

Prolog Program to Implement Best First Search Algorithm

1. Aim

To write and execute a Prolog program to implement the Best First Search algorithm on a graph, using a heuristic value to guide the search from a start node to a goal node.

2. Objective

- To understand heuristic-guided search as opposed to blind search.
- To represent a graph using edge facts and heuristic values for each node.
- To implement a rule that always expands the neighbor with the lowest heuristic value.
- To retrieve the path found by the search from a start node to a goal node.

3. Algorithm

1. Start the program.
2. Create a Prolog database using edge/2 facts to represent the graph and h/2 facts for heuristic values.
3. Define best_first/3: if the current node is the goal, the path is complete.
4. Otherwise, find all neighbors of the current node along with their heuristic values.
5. Sort the neighbors by heuristic value in ascending order.
6. Recursively continue the search from the neighbor with the lowest heuristic value.
7. Enter the start and goal nodes as a query.
8. Display the resulting path from start to goal.
9. Stop the program.

4. Flowchart



END

5. Code with Output

Code:

```
% -----  
% Graph and heuristic values  
% -----  
edge(a, b).  
edge(a, c).  
edge(b, d).  
edge(c, d).  
edge(d, e).  
  
h(a, 10).  
h(b, 8).  
h(c, 5).  
h(d, 3).  
h(e, 0).  
  
% -----  
% Best First Search  
% -----  
best_first(Goal, Goal, [Goal]) :- !.  
best_first(Start, Goal, [Start | Path]) :-  
    findall(N-H, (edge(Start, N), h(N, H)), Neighbors),  
    sort(2, @=<, Neighbors, [Best-_|_]),  
    best_first(Best, Goal, Path).
```

Output:

```
?- best_first(a, e, Path).  
Path = [a, c, d, e].
```

6. Result

The Prolog program was successfully implemented to perform a Best First Search on a graph, greedily choosing the lowest-heuristic neighbor at each step and returning the resulting path.

7. Conclusion

Thus, the Best First Search algorithm was successfully implemented using Prolog. The experiment demonstrates heuristic-guided search, where sort/4 orders candidate neighbors by an evaluation function to direct the search toward the goal more efficiently than blind search.